Project Architecture & Execution Plan: Ultra-Low Latency Conversational AI Voice Agent

---

1. Executive Summary & Core Objective

This document outlines the comprehensive technical and operational blueprint for deploying a 100% local, ultra-low-latency voice AI agent specifically designed for the hospitality sector in Nepal. The objective is to replace expensive, latency-heavy cloud telephony APIs with a localized system that answers inbound phone calls, processes orders in fluid "Nepanglish" (Nepali-English code-switching), and bypasses the steep 15% to 25% commission fees charged by third-party food delivery aggregators.

By leveraging an entirely local, open-source technology stack deployed on edge hardware, the system neutralizes cloud computing expenditures, mitigates the impacts of severe local labor shortages, and fundamentally transforms the direct-to-consumer order pipeline for urban restaurants.

2. Open-Source "Nepanglish" AI Pipeline

The foundation of this architecture is built on state-of-the-art open-source models optimized for edge deployment. This ensures that processing remains completely localized, achieving sub-800ms response latencies and zero per-minute cloud API costs.

Pipeline Layer	Technology Choice	Strategic Rationale & Functionality
Voice Activity Detection (VAD)	Silero VAD	Ultra-lightweight with minimal computational overhead. Accurately ignores Kathmandu street traffic and background restaurant chatter to determine the exact

Pipeline Layer	Technology Choice	Strategic Rationale & Functionality
		millisecond human speech ends.
Automatic Speech Recognition (ASR)	Whisper.cpp / Faster-Whisper	Transcribes 8kHz narrowband GSM audio. Supports quantized models capable of parsing highly mixed Romanized Nepali and English terms natively.
Large Language Model (LLM)	Qwen2.5 (3B/7B) or Llama 3.2 (3B)	Handles complex dialogue logic, dynamic menu parsing, and contextual intent extraction. Exceptional multilingual understanding for parsing mixed syntax (e.g., "Euta mixed pizza ra dui ota cold coffee").
Text-to-Speech (TTS)	Piper TTS / Kokoro	Generates human-like voice responses locally with extreme speed. Can be mapped to pronounce localized terms (e.g., "Choila", "Boudha") accurately.
Backend Framework	Python (FastAPI) & SQLite	High-performance, asynchronous orchestration

Pipeline Layer	Technology Choice	Strategic Rationale & Functionality
		of audio chunking, model inference pipelines, state management, and localized menu database storage.

3. Phase 1: Local Laptop Verification Setup (Zero-Cost GSM Prototype)

To bypass the infrastructural hurdle of acquiring expensive and restricted SIP trunks in Nepal, the system utilizes a custom GSM Gateway built using standard consumer hardware.

Telephony Hardware & Driver Configuration

- **Hardware:** Huawei 3G/4G USB Modem (e.g., Huawei E173, E1752) paired with an active NTC or Ncell physical SIM card.
- **Mode Switching:** The modem must be forced to drop its storage partition and expose internal serial interfaces.

```
# Install dependencies
sudo apt-get install usb-modeswitch asterisk

# Switch modem to diagnostic/serial mode
sudo usb_modeswitch -v 0x12d1 -p 0x140c -M
"55534243123456780000000000000011062000000100000000000000000000"
```

Asterisk & chan_dongle Architecture

The Asterisk PBX interfaces directly with the USB modem using the chan_dongle channel driver, processing cellular audio directly into the digital VoIP environment.

```
# /etc/asterisk/dongle.conf
[dongle0]
audio=/dev/ttyUSB1
data=/dev/ttyUSB2
context=inbound-calls
imei=YOUR_MODEM_IMEI
imsi=YOUR_SIM_IMSI

# /etc/asterisk/extensions.conf
[inbound-calls]
exten => _X.,1,Answer()
same => n,EAGI(python3 /path/to/voice_agent_bridge.py)
same => n,Hangup()
```

4. Handling "Nepanglish", Code-Switching & Menus

The target demographic utilizes a highly fluid code-switching dialect. The language model is strictly grounded using specific system prompts to parse romanized Nepali mixed with English culinary terminology.

```
SYSTEM_PROMPT = """
You are "Maya", the AI receptionist for Everest Burger, Kathmandu.
You speak fluent 'Nepanglish' (a natural mix of romanized Nepali and English).

Core Rules:
1. Speak warmly, briefly, and politely (Use "Hajur", "Namaste", "Bhannus na").
2. Match the caller's language: If they speak English, respond in English. If they mix Nepali and English, mix naturally.
3. Only confirm items present in the menu. Always specify quantity.
4. Output concise sentences to maintain fast voice delivery.
"""
```

Voice Humanization: The LLM is instructed to generate conversational fillers (e.g., *"Umm", "Hajur", "Ek chin hai..."*) to mask underlying computational latency and reinforce the illusion of human interaction.

5. Multi-Modal Address Resolution (SMS GPS Fallback)

Because formalized street addresses are largely non-existent in Kathmandu, the system utilizes a multimodal SMS fallback mechanism to guarantee delivery success without expensive mapping APIs.

1. Upon order confirmation, the backend executes an AT command (AT+CMGS) via the modem's data port (/dev/ttyUSB2).
2. The caller instantly receives an SMS: *"Tap link to confirm delivery location: http://<local_ip>/loc/ord_9821"* while still on the call.
3. The user taps the link to open a lightweight webpage that requests GPS permission, drops a pin, and sends the exact latitude and longitude back to the local Python backend.

6. Production Roadmap & Deployment Execution

Phase 1: Proof of Concept (Laptop Verification)

- Procure a standard PC (Intel i5/i7) and a Huawei GSM USB Modem from local tech markets in Kathmandu.
- Install Ubuntu, Asterisk, and the Python FastAPI backend on the edge node.
- Run localized tests adjusting the VAD thresholds to ignore ambient noise.

Phase 2: Administrative Dashboard Initialization

- Develop a responsive **React.js + Tailwind CSS** dashboard served locally.
- Include modules for **Menu Management** (CRUD operations directly updating the SQLite DB) and a **Live Call/Kanban Board** (WebSocket-powered view of active calls and GPS pins).
- Enforce a strict **Zero Payment Integration (Cash on Delivery)** model to bypass regulatory friction.

Phase 3: Scale to Production-Grade Hardware

- Migrate from USB dongles to dedicated Multi-Port GSM Gateways (e.g., OpenVox or Dinstar) connected via standard LAN Ethernet for increased stability and multi-line capacity.

- Deploy robust local edge servers in client restaurants for sustained operations.

Document finalized and structured for direct engineering deployment.